

AI TEAM · ONBOARDING

AI Engineering Training Plan

A structured, project-based roadmap taking a new team member from zero background to actively contributing to our AI workflows — with real understanding of what they build.

PREPARED FOR New AI Engineering Intern

DURATION ~8–12 weeks · 10 phases · project-gated

APPROACH Build to learn · specify before you generate · own correctness

Program Overview

Goal. Take someone with little or no AI background to the point where they can read our architectures, reason about our workflows, and implement real tasks (mostly via Claude Code) with genuine understanding of what they are building.

How to use this. Work phase by phase. Do not just watch — every phase ends with a small “prove it” build. If you cannot do the build, you are not done with the phase. Treat the week counts as a guide, not a rule.

The running thread. Reading architectures and using Claude Code well starts in Phase 3 and grows the whole way through — it is not a separate module bolted on at the end.

Roadmap at a Glance

0	Setup & Mindset Environment, Git, and how we actually work	2–3 days
1	Python Foundations + OOP Read Python fluently; classes, NumPy, Pandas	1 week
2	ML Fundamentals & Core Terminology Vocabulary and intuition, not production models	1–1.5 weeks
3	Deep Learning, In Depth From words to the machine — backprop from scratch	2 weeks
4	LLM Basics → Architecture → How They Work Tokens, embeddings, transformers, attention	1.5–2 weeks
5	Working With LLMs in Practice APIs, prompting, RAG and grounding	1 week
6	Frameworks, Agents & Multi-Agent (LangGraph) The orchestration layer we build systems in	2 weeks
7	Serving Models with FastAPI Turn an agent into something callable	1 week
8	Architecture Thinking + Claude Code The skill that makes you useful on the team	ongoing
9	AI Security: Injection & Backend Validation The line between a toy and something we ship	1 week

Before any AI, get the basic developer muscle in place.

- **Environment:** Python 3.11+, VS Code, Git. Create and use virtual environments (venv) and pip.
- **Git basics:** clone, branch, commit, push, pull request. You will live in these.
- **How we work:** we ship most development through Claude Code, but the human owns the architecture and the plan. The AI writes code; you decide what is correct. Keep that framing from day one.

RESOURCES

- ▶ Git — GeeksforGeeks Git tutorial: [geeksforgeeks.org/git-tutorial](https://www.geeksforgeeks.org/git-tutorial)
- ▶ Python setup + venv — official docs: docs.python.org/3/tutorial/venv.html
- ▶ freeCodeCamp “Git & GitHub for Beginners” (YouTube, ~1 hr)

CHECKPOINT

Create a repo, make a branch, commit a “hello world” Python script, and open a pull request.

You need to *read* Python fluently even if Claude Code writes most of it. Focus on comprehension over memorization.

- Core: variables, types, control flow, functions, list/dict comprehensions
- Data structures: lists, dicts, sets, tuples — and when to use each
- **OOP:** classes, objects, `__init__`, inheritance, methods vs. attributes, `self`. This matters because AI libraries are class-heavy (models are objects).
- Practical: file I/O, virtual environments, pip, reading a `requirements.txt`
- Comfortable with NumPy (arrays) and Pandas (dataframes) at a basic level

RESOURCES

- ▶ Python OOP — GeeksforGeeks: [geeksforgeeks.org/python-oops-concepts](https://www.geeksforgeeks.org/python-oops-concepts)
- ▶ freeCodeCamp “Python for Beginners” full course (YouTube, ~4 hrs)
- ▶ Corey Schafer’s Python OOP playlist (YouTube) — clearest explanation of classes/inheritance out there
- ▶ NumPy + Pandas — GeeksforGeeks NumPy and Pandas tutorials

CHECKPOINT

Write a small class (e.g., `BankAccount` or `Task`) with methods and inheritance. Then load a CSV with Pandas and compute a couple of summary stats.

The point here is **vocabulary and intuition**, not building production models. When someone says “we’re overfitting” or “the embeddings are off,” you should know what that means.

- What ML *is*: learning patterns from data vs. writing rules
- Supervised vs. unsupervised vs. reinforcement learning
- Core terms: features, labels, train/validation/test split, weights, loss, gradient descent, epoch
- Overfitting vs. underfitting, bias/variance, regularization (concept level)
- A few classic algorithms conceptually: linear/logistic regression, decision trees, k-means
- Evaluation: accuracy, precision, recall, F1 (what they mean, not the formulas by heart)
- **Embeddings** — introduce the concept here; it becomes central for LLMs/RAG later.

RESOURCES

- ▶ GeeksforGeeks “Machine Learning” hub: [geeksforgeeks.org/machine-learning](https://www.geeksforgeeks.org/machine-learning)
- ▶ **StatQuest with Josh Starmer** (YouTube) — the single best channel for ML intuition. ML fundamentals, Bias/Variance, Gradient Descent, Decision Trees. Do not skip StatQuest.
- ▶ Google’s “Machine Learning Crash Course” (free) — especially the terminology glossary

CHECKPOINT

In your own words (short doc or Loom), explain to the team: what overfitting is, what a train/test split is for, and what an embedding roughly represents. Teaching it back is the test.

This is where you go from “knows the words” to “understands the machine.” Spend real time here — everything after (LLMs, agents) sits on this.

- Neurons, layers, activation functions
- Forward pass, loss, **backpropagation**, gradient descent (now the mechanical version)
- Training loop: batches, epochs, learning rate, optimizers
- Key architectures conceptually: MLPs, CNNs (vision), RNNs (sequence) — and *why* each shape suits its data
- Intro to a framework: **PyTorch** basics (tensors, `nn.Module`, a training loop)
- Light math alongside: derivatives (intuition), dot products / matrix multiply, basic probability. Karpathy assumes only “a vague recollection of high-school calculus.”

RESOURCES

- ▶ **Andrej Karpathy** — “**Neural Networks: Zero to Hero**” (free, builds NNs from scratch up to a GPT): karpathy.ai/zero-to-hero.html — the centerpiece of the whole roadmap. Code alongside him.
- ▶ Notebooks / repo: github.com/karpathy/nn-zero-to-hero
- ▶ 3Blue1Brown “Neural Networks” series (YouTube) — the best visual intuition for backprop. Watch before or during Karpathy.
- ▶ Deep Learning terms — GeeksforGeeks: [geeksforgeeks.org/deep-learning-tutorial](https://www.geeksforgeeks.org/deep-learning-tutorial)

CONTINUOUS THREAD STARTS HERE

From this phase on, before writing code, sketch a one-paragraph “what are the components and how do they connect.” Then let Claude Code implement it. Get used to *specifying before generating*.

CHECKPOINT

Complete Karpathy’s `micrograd` and at least the first `makemore` video — build a tiny neural net and a character-level model from scratch. That single exercise proves you understand backprop.

Now connect deep learning to the thing we actually build with.

- What an LLM is: next-token prediction at scale
- **Tokenization** and embeddings (why spelling bugs happen)
- **The Transformer** and **attention** — self-attention, queries/keys/values, positional encoding, why attention replaced RNNs
- Pretraining vs. fine-tuning vs. instruction-tuning vs. RLHF (concept level)
- Inference knobs you'll tune in practice: temperature, top-p, context window, max tokens
- Why LLMs hallucinate, and what "grounding" means (sets up RAG and security later)

RESOURCES

- ▶ Karpathy's "Let's build GPT" and "Let's build the GPT Tokenizer" (within Zero to Hero) — the payoff of Phase 3
- ▶ 3Blue1Brown "But what is a GPT?" / attention videos — best visual explanation of transformers
- ▶ The Illustrated Transformer (Jay Alammar, blog) — the classic diagram-driven walkthrough
- ▶ GeeksforGeeks "Transformer" and "Large Language Model" articles for terminology reinforcement

CHECKPOINT

Draw the transformer data flow: raw text → tokens → embeddings → attention → output token, and explain each arrow. If you can whiteboard it, you understand it.

Bridge from theory to day-to-day: calling models, prompting well, and grounding them in real data.

- Calling an LLM via API (messages, system prompt, tools)
- Prompt engineering fundamentals: clear instructions, examples, structured (JSON) output, step-by-step reasoning
- **RAG (Retrieval-Augmented Generation)**: what it is, why it grounds models, the pipeline (chunk → embed → store in a vector DB → retrieve → stuff into prompt)
- Vector databases at a basic level (e.g., FAISS / Chroma)

RESOURCES

- ▶ Anthropic prompt engineering docs: docs.claude.com/.../prompt-engineering/overview
- ▶ GeeksforGeeks "Retrieval Augmented Generation (RAG)" article for the concept + pipeline
- ▶ A short freeCodeCamp or official RAG tutorial (YouTube) to build one end to end

CHECKPOINT

Build a tiny RAG demo — feed in a few internal docs (or any PDFs) and answer questions grounded in them. This is the seed of a lot of our real work.

Now the orchestration layer we actually build systems in.

- LangChain basics (just enough): what a “chain” is, prompts, tools, why frameworks exist
- **What an agent is:** an LLM in a loop that can reason, call tools, and decide next steps (the ReAct pattern)
- **LangGraph:** model workflows as a graph of nodes (agents/tools) and edges (decisions), with shared state, loops, and checkpoints — so multiple agents collaborate reliably instead of a brittle single chain
- **Multi-agent patterns:** supervisor (one agent delegates to workers) vs. network/decentralized (peer-to-peer). Know when to use which.
- Demo → production concepts: shared state schema design, checkpointing (recover after a crash), and human-in-the-loop (`interrupt()`)

RESOURCES

- ▶ Official LangGraph docs + quickstart (start here — the framework moves fast, so trust the docs over old blog posts)
- ▶ DataCamp “Multi-Agent Systems with LangGraph” (~2 hr) — supervisor and network patterns hands-on
- ▶ freeCodeCamp “Build Your First Multi-Agent AI System in Python and LangGraph” — builds it with and without the framework so you see what LangGraph buys you

LESSON TO INTERNALIZE EARLY

State schema design is the decision that ages worst in LangGraph projects. Don't dump everything into one flat dict “to fix later.”

CHECKPOINT

Build a 2–3 agent LangGraph system with a supervisor delegating to workers (e.g., a “researcher” + a “writer”). Add one tool call and one conditional edge.

Turn a working agent/model into something the rest of a system can call.

- REST API basics: endpoints, request/response, status codes, JSON
- **FastAPI:** routes, path/query params, **Pydantic models** for request validation (this Pydantic habit pays off directly in the security phase)
- Async basics (FastAPI is async-first)
- Wrapping an LLM/agent call behind an endpoint; streaming responses (nice-to-have)
- **Docker basics:** containerize the API so it runs anywhere (concept + a simple Dockerfile)

RESOURCES

- ▶ FastAPI official tutorial (excellent, beginner-friendly): fastapi.tiangolo.com/tutorial
- ▶ GeeksforGeeks “FastAPI” tutorial for a quick orientation
- ▶ freeCodeCamp FastAPI course (YouTube) for an end-to-end build

CHECKPOINT

Wrap your Phase 6 agent in a FastAPI endpoint with a Pydantic-validated request body. Bonus: containerize it with a Dockerfile.

You've been doing the "spec before generate" thread since Phase 3. Now make it deliberate. This is the skill that actually makes you useful on our team.

- Read one of our existing project architectures end to end; write a paragraph explaining how data flows through it
- Practice our loop: **understand the problem** → **sketch the architecture** → **break into tasks** → **drive Claude Code to implement** → **review the output critically** → **integrate**
- Give Claude Code good specs: clear goals, constraints, file structure, examples. Vague prompts get vague code.
- Learn to *review* generated code — you own correctness, not the model

RESOURCES

- ▶ Our internal repos and architecture docs (ask the team for 1–2 representative ones)
- ▶ Anthropic prompt engineering docs (same as Phase 5) apply directly to prompting Claude Code well

CHECKPOINT

Take a small real (or realistic) internal task, write the architecture/plan yourself first, then implement it with Claude Code, and walk a teammate through both the plan and the result.

Deliberately last, because it only makes sense once you understand LLMs, agents, and APIs. Increasingly, this is the difference between a toy and something we can ship.

- **The core problem:** LLMs can't reliably tell *data* from *instructions*. A system prompt is **not** a security boundary.
- **Prompt injection**
 - *Direct*: user types "ignore your instructions and..."
 - *Indirect*: malicious instructions hidden in content the model reads — a PDF, webpage, support ticket, or repo. This is the dangerous one for RAG/agent systems.
- **Grounding** (RAG) reduces hallucination but does **not** by itself stop injection — know the difference.
- **Defenses (where backend validation lives):**
 - Never trust model output for privileged actions — validate on the backend before executing
 - Constrain tools and permissions (least privilege for agents)
 - Validate and sanitize inputs *and* outputs; use structured output + Pydantic schemas as a gate
 - Human-in-the-loop for high-risk actions; treat retrieved/external content as untrusted

RESOURCES

- ▶ **OWASP Top 10 for LLM Applications (2025)** — the canonical reference. Start with LLM01: Prompt Injection: genai.owasp.org/llm-top-10
- ▶ Practitioner blog posts on prompt injection for intuition, plus the OWASP mitigation cheat sheets
- ▶ GeeksforGeeks articles on input validation / API security for backend-validation fundamentals

CHECKPOINT

Take your Phase 7 API and add: (1) input validation via Pydantic, (2) a rule that no model output can trigger a privileged action without a backend check, and (3) a short threat-model note on where indirect injection could enter your RAG demo.

Final Graduation Project

Combine it all into one small but real system.

The Capstone

A FastAPI service exposing a LangGraph multi-agent workflow that answers questions grounded in a document set (RAG), with Pydantic input validation and at least one backend guardrail against prompt injection — built by planning the architecture first, then implementing with Claude Code.

If they can build, explain, and defend the design choices of that project, they're ready to work on our real workflows.

Quick Reference — Anchor Resources

Area	Primary resource
Python / OOP	GeeksforGeeks Python OOP; Corey Schafer (YouTube)
ML intuition	StatQuest (YouTube); Google ML Crash Course
Deep Learning	Karpathy, Neural Networks: Zero to Hero ; 3Blue1Brown
Transformers / LLMs	Karpathy "Let's build GPT"; 3Blue1Brown; The Illustrated Transformer
Prompting / RAG	Anthropic prompt docs; GeeksforGeeks RAG
Agents / LangGraph	Official LangGraph docs; DataCamp course; freeCodeCamp article
Serving	FastAPI official tutorial
Security	OWASP Top 10 for LLM Applications (2025)

Two habits matter more than any single resource: (1) always build the small thing at the end of each phase, and (2) sketch the architecture before you let Claude Code write the code.